


University
Politehnica
of Bucharest







# PL/SQL (III)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



# Overview

- Composite data types



# Composite data types

- A composite data type stores values that have internal components
- PL/SQL lets you define two kinds of composite data types:
  - Collection
  - Record
- A composite variable
  - Can be passed to subprograms as parameters
  - Its internal components can be accessed individually
- Internal components can be
  - Scalar
  - Composite



# Collections

- The internal components of a collection
  - Have the same data type
  - Are called elements
- Elements of a collection variable can be accessed by its unique index, e.g., **variable_name(index)**
- Collections variables can be
  - Copies of existing composite variables:
    - Created using the attribute %TYPE
    - Inherite the only structure , not the values
  - User defined:
    - Define a collection type
    - Create a variable of that type



# Records

- In a record, the internal components
  - Can have different data types
  - Are called fields
- Each field of a record variable can be accessed by its name, e.g., variable_name.field_name.
- Record variables can be
  - Copies of exisitng composite variables:
    - Created using the attribute using %ROWTYPE or %TYPE
    - Inherite the only structure , not the values
  - User defined:
    - Define a RECORD type
    - Create a variable of that type



# Records

- A RECORD type defined in a PL/SQL block is
  - A local type
  - Available only in the block
  - Stored in the database only if the block is in a standalone or package subprogram
- A RECORD type defined in a package specification
  - Is a public item
  - Can be referenced from outside the package by qualifying it with the package name (package_name.type_name)
  - Is stored in the database until you drop the package with the DROP PACKAGE statement
- A RECORD type cannot be created at schema level (a RECORD type cannot be an ADT attribute data type)



# Records

- To define a RECORD type specify the record's name and define its fields
- To define a field specify the filed's name and a its data type
- The initial value of a field is NULL by default
- The NOT NULL constraint can be specified for a field (a non-NULL initial value must also be specified)
- A non-NULL initial value is optional without the NOT NULL constraint
- A RECORD type defined in a package specification is incompatible with an identically defined local RECORD type
- The initial value of each field is NULL for a record variable
  - Of a RECORD type (unless you specify a different initial value for it when you define the type)
  - Declared with %ROWTYPE or %TYPE (the variable does not inherit the initial value of the referenced item)



# Records

- Records cannot be tested natively for nullity, equality, or inequality

- These BOOLEAN expressions are illegal:
  - My_Record IS NULL
  - My_Record_1 = My_Record_2
  - My_Record_1 > My_Record_2



# Records Example

```
set serveroutput on
DECLARE
  TYPE rec1 IS RECORD(
    id countries.country_id%TYPE,
    name varchar2(20),
    reg regions%ROWTYPE
  );
  r1 rec1;
  r2 countries%ROWTYPE;
  r3 countries%ROWTYPE;
BEGIN
  SELECT * INTO r1.reg FROM regions WHERE lower(region_name) LIKE '%rop_';

  r1.id := 'RO';
  r1.name := 'Romania';

  INSERT INTO countries VALUES(r1.id, r1.name, r1.reg.region_id);

  SELECT * INTO r2 FROM countries WHERE country_id = r1.id;

  r2.country_id := 'ME';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  INSERT INTO countries VALUES r2;

  r2.country_id := 'MD';
  r2.country_name := 'Moldova';
  r2.region_id := r1.reg.region_id;

  UPDATE countries SET ROW = r2 WHERE country_id = 'ME'
  RETURNING country_id, country_name, region_id INTO r3;

  DELETE FROM countries WHERE country_id = 'MD'
  RETURNING country_id, country_name, region_id INTO r3;

END;
/
```



# Records

- These restrictions apply to record inserts and updates:
  - Record variables are allowed only in these places:
    - On the right side of the SET clause in an UPDATE statement
    - In the VALUES clause of an INSERT statement
    - In the INTO subclause of a RETURNING clause
  - Record variables are not allowed in a SELECT list, WHERE clause, GROUP BY clause, or ORDER BY clause.
  - The keyword ROW is allowed only on the left side of a SET clause
  - ROW cannot be used with a subquery
  - Only one SET clause is allowed if ROW is used in an UPDATE statement



# Records

- These restrictions apply to record inserts and updates:
  - If the VALUES clause of an INSERT statement contains a record variable, no other variable or value is allowed in the clause
  - If the INTO subclause of a RETURNING clause contains a record variable, no other variable or value is allowed in the subclause
  - These are not supported:
    - Nested RECORD types
    - Functions that return a RECORD type
    - Record inserts and updates using the EXECUTE IMMEDIATE statement



# Composite data types

- Collection have one dimension
- A multidimensional collection can be modeled by using a collection whose elements are collections
- Developers can create
  - A collection of records
    - Define a collection that has elements records
    - Accessing an item: **variable_name(index).field_name**
  - A record that contains collections
    - Define a record that has items collections
    - Accessing a field: **variable_name.field_name(index)**
  - A collection of collections:
    - Define a collection that has items a collection
    - Access an item: **variable_name(index_1)(index_2)…(index_n)**



# Collections types

- PL/SQL has three collection types
  - Associative array (index-by table)
  - VARRAY (variable-size array)
  - Nested table

| Collection Type | Number of Elements | Index Type | Dense or Sparse | Uninitialized Status | Where Defined | Can Be an ADT Type Attribute |
|---|---|---|---|---|---|---|
| Associative array (or index-by table) | Unspecified | String or PLS_INTEGER / BINARY_INTEGER | Either | Empty | In PL/SQL block or package | No |
| VARRAY (variable-size array) | Specified | Integer | Always dense | Null | In PL/SQL block or package or at schema level | Only if defined at schema level |
| Nested table | Unspecified | Integer | Starts dense, can become sparse | Null | In PL/SQL block or package or at schema level | Only if defined at schema level |



# Collection types

- Number of Elements?
  - Specified:  the maximum number of elements in the collection
  - Unspecified: the maximum number of elements in the collection is the upper limit of the index type

- Dense or Sparse?
  - A dense collection
    - has no gaps between elements
    - every element between the first and last element is defined
    - has a value (the value can be NULL unless the element has a NOT NULL constraint).
  - A sparse collection has gaps between elements.



# Collection types Uninitialized Status

- An empty collection exists but has no elements
  - The EXTEND method must be invoked to add elements to an empty collection
- A null collection (also called an atomically null collection) does not exist.
  - The collection must be initialized to change it from a null collection to an existing collection
  - Initialization:
    - Making it empty
    - Assigning a non-NULL value to it
  - The EXTEND method cannot be used to initialize a null collection



# Collection types Where defined

- A collection type defined in a PL/SQL block:
  - Is a local type
  - Is available only in the block
  - Is stored in the database only if the block is in a standalone or package subprogram

- A collection type defined in a package specification
  - Is a public item
  - Can be referenced from outside the package by qualifying it with the package name (package_name.type_name)
  - Is stored in the database until you drop the package

# Collection types
# Where defined

- A collection type defined at schema level
  - Is a standalone type.
  - Can be created with the CREATE [OR REPLACE] TYPE Statement
  - Is stored in the database until you drop it with the DROP TYPE Statement

- Note:
  - A collection type defined in a package specification is incompatible with an identically defined local or standalone collection type



# Collection types
# ADT Attribute Data Type

- To be an (**Abstract Data Type**) Attribute Data Type, a collection type must be a standalone collection type

- Translating Non-PL/SQL Composite Types to PL/SQL Composite Types:

| Non-PL/SQL Composite Type | Equivalent PL/SQL Composite Type |
|---|---|
| Hash table | Associative array |
| Unordered table | Associative array |
| Set | Nested table |
| Bag | Nested table |
| Array | VARRAY |



# Associative Arrays

- An associative array (formerly called PL/SQL table or index-by table)
  - Is a set of key-value pairs
  - Each key is a unique index
- The index is used to locate the associated value with the syntax **variable_name(index)**
- The data type of index can be
  - A string type (VARCHAR2, VARCHAR, STRING, or LONG)
  - PLS_INTEGER/BINARY_INTEGER



# Associative Arrays

- Indexes
  - Are stored in sort order
  - Are not created in order

- The sort order is determined by the initialization parame ters NLS_SORT and NLS_COMP for string types

- National Language Support (NLS) parameters such as NLS_SORT, NLS_COMP, and NLS_DATE_FORMAT affect associative arrays indexed by string



# Associative Arrays

- An associative array, like a database table:
  - Is empty (but not null) until populating it
  - Can hold an unspecified number of elements (accessible without knowing their positions)

- An associative array, unlike a database table:
  - Does not need disk space or network operations
  - Cannot be manipulated with DML statements



# Associative Arrays

- An associative array is appropriate for:

1. A relatively small lookup table constructed in memory each time
   - The subprogram is invoked
   - The package that declares it is initialized

2. To pass collections to and from the database server
   - Declare formal subprogram parameters of associative array types
   - Bind the host arrays to the corresponding actual parameters - use Oracle Call Interface (OCI) or an Oracle precompiler
   - PL/SQL automatically converts between host arrays and associative arrays indexed by PLS_INTEGER
   - You cannot bind an associative array indexed by VARCHAR.



# Associative Arrays

- The associative array type cannot be declared at schema level
- An associative array is intended for temporary data storage
- To make an associative array persistent for the life of a database session
  - Declare it in a package specification
  - Populate it in the package body



# Associative Arrays

- To pass an associative array variable as a parameter to a standalone subprogram:
  - The type of that variable must be declared in a package specification.
  - Doing so makes the type available to
    - The invoked subprogram (which declares a formal parameter of that type)
    - The invoking subprogram or anonymous block (which declares and passes the variable of that type)
- The most efficient way to pass collections to and from the database server is to use associative arrays with
  - FORALL statement
  - BULK COLLECT clause



# Associative Arrays Example

```
 1 set serveroutput on
 2 DECLARE
 3   TYPE rec IS RECORD(
 4     country_id countries.country_id%TYPE,
 5     country_name countries.country_name%TYPE,
 6     region_id countries.region_id%TYPE
 7   );
 8   TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
 9
10     v aa; -- create a null associative array
11   r rec;
12
13   idx PLS_INTEGER;
14
15   TYPE idxs_aa IS TABLE OF PLS_INTEGER INDEX BY PLS_INTEGER;
16   idxs idxs_aa;
17   i PLS_INTEGER := 1;
18 BEGIN
19   r.country_id := 'RO';
20   r.country_name := 'Romania';
21   r.region_id := 1;
22
23   v := aa(1 => r, 2 => rec('MD', 'Moldova', 1));
24
25   v(5).country_id := 'BG';
26   v(5).country_name := 'Bulgaria';
27   v(5).region_id := 1;
28
29   idx := v.FIRST;
30   WHILE idx IS NOT NULL
31   LOOP
32     dbms_output.put_line(idx||' '||v(idx).country_id||' '
33                                  ||v(idx).country_name||' '||v(idx).region_id);
34     idxs(i) := idx;
35     idx := v.next(idx);
36     i := i + 1;
37   END LOOP;
38
39   FORALL j IN VALUES OF idxs INSERT INTO COUNTRIES VALUES v(j);
40
41 END;
42 /
```



# Associative Arrays Example

```
 1 DECLARE
 2   TYPE rec IS RECORD(
 3     country_id countries.country_id%TYPE,
 4     country_name countries.country_name%TYPE,
 5     region_id countries.region_id%TYPE
 6   );
 7
 8   TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
 9   v aa := aa(); -- create an empty associative array;
10   r rec;
11   idx PLS_INTEGER;
12 BEGIN
13   r.country_id := 'RO';
14   r.country_name := 'Romania';
15   r.region_id := 1;
16
17   v(1) := r;
18   v(2) := rec('MD', 'Moldova', 1);
19
20   v(3).country_id := 'BG';
21   v(3).country_name := 'Bulgaria';
22   v(3).region_id := 1;
23
24   SELECT * BULK COLLECT INTO v from countries;
25
26   idx := v.FIRST;
27   WHILE idx IS NOT NULL
28   LOOP
29     dbms_output.put_line(idx||' '||v(idx).country_id
30                          ||' '||v(idx).country_name||' '||v(idx).region_id);
31     idx := v.next(idx);
32   END LOOP;
33 END;
34 /
```



# VARRAYS

- A varray (variable-size array) is an array whose number of elements can vary from zero (empty) to the declared maximum size
- Use the syntax **variable_name(index)** to access an element of a varray variable,
- The lower bound of index is 1
- The upper bound
  - Is the current number of elements
  - Changes as you add or delete elements
  - Cannot exceed the maximum size



# VARRAYS

- Indexes and element order remain stable when
  - A varray is stored
  - A varray is retrieved
- The database stores a varray variable as a single object
- A varray resides
  - Inside the table of which it is a column for a varray variable less than 4 KB
  - Outside the table but in the same tablespace for a varray variable less than 4 KB



# VARRAYS

- An uninitialized varray variable is a null collection
- A varray variable must be initialized by
  - Making it empty
  - Assigning a non-NULL value to it
- A varray is appropriate when:
  - The maximum number of needed elements is known
  - The elements are accessed sequentially
- A varray might be impractical for large numbers of elements because all elements must be stored or retrieved at the same time



# VARRAYS Example

```
 1 DECLARE
 2   TYPE vec1 IS VARRAY(150) OF countries.country_id%type;
 3   v1 vec1 := vec1('RO', 'MD');
 4   TYPE vec2 IS VARRAY(150) OF countries.country_name%type;
 5   v2 vec2 := vec2('Romania', 'Moldova');
 6 BEGIN
 7   FORALL i IN v1.FIRST .. v1.LAST
 8     INSERT INTO countries(country_id) VALUES(v1(i));
 9
10   FORALL i IN v2.FIRST .. v2.LAST
11     UPDATE countries SET country_name=v2(i), region_id=1
12     WHERE country_id = v1(i);
13
14   SELECT country_id, country_name
15   BULK COLLECT INTO v1, v2
16   FROM countries
17   WHERE country_id NOT IN (SELECT DISTINCT country_id
18                            FROM locations
19                            WHERE country_id IS NOT NULL);
20
21   FOR i IN v1.FIRST .. v1.LAST
22   LOOP
23     dbms_output.put_line(v1(i)||' '||v2(i));
24   END LOOP;
25
26   FORALL i IN v1.FIRST .. v1.LAST
27     DELETE countries WHERE country_id = v1(i);
28
29 END;
30 /
```



# Nested Tables

- A nested table is a column type that stores an unspecified number of rows in no particular order
- When retrieving a nested table value from the database into a PL/SQL nested table variable:
  - PL/SQL gives the rows consecutive indexes
  - These indexes start at 1
- The indexes help to access the individual rows of the nested table variable
- The syntax is variable_name(index)



# Nested Tables

- A nested table's indexes and row order might not remain stable when
  - Storing the nested table  to the database
  - Retrieving the nested table from the database
- The amount of memory that a nested table variable occupies can
  - Increase dynamically as elements are added
  - Decrease dynamically as elements are delete
- An uninitialized nested table variable is a null collection
- A nested table is initialized by:
  - Making it empty
  - Assigning a non-NULL value to it

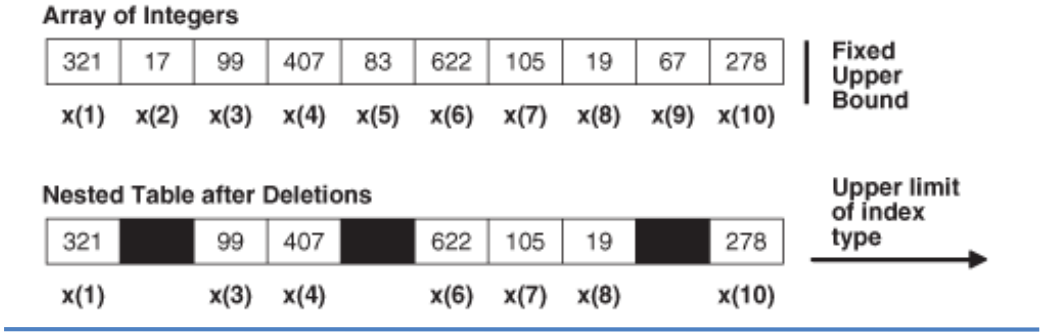
# Nested Tables

- Conceptually a nested table is like a one-dimensional array with an arbitrary number of elements

- However, a nested table differs from an array in these important ways:

- The size of a nested table can increase dynamically:
  - An array has a declared number of elements
  - A nested table does not have a declared number of elements

- A nested table can become sparse
  - An array is always dense
  - A nested table is dense initially
  - A nested table become sparse as elements are delete





# Nested Tables

- A nested table is appropriate when
  - The number of elements is not set
  - Index values are not consecutive
  - To delete or update some elements, but not all elements simultaneously



# Nested Tables Example

```
DECLARE
  TYPE tbl1 IS TABLE OF countries.country_id%type;
  t1 tbl1 := tbl1('RO', 'MD');
  TYPE tbl2 IS TABLE OF countries.country_name%type;
  t2 tbl2 := tbl2('Romania', 'Moldova');
BEGIN
  FORALL i IN INDICES OF t1 INSERT INTO countries(country_id) VALUES(t1(i));

  FORALL i IN INDICES OF t2 UPDATE countries SET country_name=t2(i), region_id=1 WHERE country_id = t1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO t1, t2 FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT NULL);

  FOR i IN t1.FIRST .. t1.LAST
  LOOP
    dbms_output.put_line(t1(i)||' '||t2(i));
  END LOOP;

  FORALL i IN INDICES OF t1 DELETE countries WHERE country_id = t1(i);

END;
/
```



# Collection Constructors

- A **collection constructor (constructor)**
  - Is a system-defined function
  - Has the same name as a collection type
  - Returns a collection of that type

```
-- declare the type
TYPE vec IS VARRAY(10) OF INTEGER;
-- initialize an empty variable
var vec := vec();
```



# Assigning Values to Collection Variables

- Constructor:
  - Invoke a constructor to create a collection
  - Assign values to the collection elements
- Assignment statement
  - To assign the value of another existing collection variable
- Subprogram:
  - Pass the collection variable  to a subprogram
  - Use the OUT or IN OUT parameter
  - Assign the value inside the subprogram
- Note:
  - A collection can be assigned to a collection variable only is they have the same data type
  - Having the same element type is not enough for an assignment



# Assigning Values to Collection Variables

- To make a collection variable null assign:
  - A NULL value
  - A NULL collection of the same data type

- To a nested table variable can be assigned:
  - The result of a SQL MULTISET operation
    - The SQL MULTISET operators combine two nested tables into a single nested table
    - The elements of the two nested tables must have comparable data types
    - MULTISET EXCEPT
    - MULTISET INTERSECT
    - MULTISET UNION
  - The result of the SQL SET function invocation
    - The SQL SET function
      - Takes a nested table argument
      - Returns a nested table of the same data type whose elements are distinct
    - The function eliminates duplicate elements



# MULTISET Examples

```
DECLARE
    TYPE t IS TABLE OF INTEGER;
    s1 t := t(1,1,2,2,3,4);
    s2 t := t(2,4,4,6,6, 8, 8);
    s3 t;
    s4 t := t(3, 4);
    card number(4);
BEGIN
    IF s3 IS NULL THEN
        dbms_output.put_line('EMPTY AND NULL');
    ELSIF s3 IS NOT NULL AND s3 IS EMPTY THEN
        dbms_output.put_line('EMPTY AND NOT NULL');
    ELSIF s3 IS NOT NULL AND s3 IS NOT EMPTY THEN
        dbms_output.put_line('NOT EMPTY AND NOT NULL');
    END IF;

    s3 := s2 MULTISET UNION S1; -- 2 4 4 6 6 8 8 1 1 2 2 3 4
    s3 := SET(s3); -- 2 4 6 8 1 3
    s3 := s1 MULTISET INTERSECT S2; -- 2 4
    s3 := s1 MULTISET EXCEPT S2; -- 1 1 2 3
    s3 := s1 MULTISET EXCEPT DISTINCT S2; -- 1 2
    s3 := s2 MULTISET EXCEPT S1; -- 4 6 6 8 8
    s3 := s2 MULTISET EXCEPT DISTINCT S1; -- 6 8

    IF s4 SUBMULTISET OF s1 THEN
        dbms_output.put_line('TRUE');
    ELSE
        dbms_output.put_line('FALSE');
    END IF;

    card := CARDINALITY(s1); -- 6
    card := CARDINALITY(SET(s1)); -- 4

    IF &VAL MEMBER OF S1 THEN
        dbms_output.put_line('TRUE');
    END IF;
END;
/
```



# Collection Methods

- A collection method is a PL/SQL subprogram
  - A function that returns information about a collection
  - A procedure that operates on a collection

- Collection methods make
  - Collections easier to use
  - Applications easier to maintain

- The syntax for a method invocation is:

**collection_name.method**



# Collection Methods

| Method | Type | Description |
|---|---|---|
| DELETE | Procedure | Deletes elements from collection. |
| TRIM | Procedure | Deletes elements from end of varray or nested table. |
| EXTEND | Procedure | Adds elements to end of varray or nested table. |
| EXISTS | Function | Returns TRUE if and only if specified element of varray or nested table exists. |
| FIRST | Function | Returns first index in collection. |
| LAST | Function | Returns last index in collection. |
| COUNT | Function | Returns number of elements in collection. |
| LIMIT | Function | Returns maximum number of elements that collection can have. |
| PRIOR | Function | Returns index that precedes specified index. |
| NEXT | Function | Returns index that succeeds specified index. |



# Collection Methods
## DELETE

- **DELETE** is a procedure that deletes elements from a collection

- This method has these forms:
  - **DELETE** deletes all elements from a collection of any type
  - This operation immediately frees the memory allocated to the deleted elements
  - From an associative array or nested table (**but not a varray**):
    - **DELETE(n)** deletes the element whose index is **n**
      - If that element exists
      - Otherwise, it does nothing
    - **DELETE(m,n)** deletes all elements whose indexes are in the range **m..n**,
      - If both **m** and **n** exist and **m <= n**
      - Otherwise, it does nothing
    - For these two forms of **DELETE**:
      - PL/SQL keeps placeholders for the deleted elements
      - The deleted elements are included in the internal size of the collection
      - A deleted element can be restored by assigning a valid value to it



# Collection Methods
## DELETE Example

```
 1 DECLARE
 2   TYPE nt is TABLE OF NUMBER(4);
 3   t nt := nt(11, 22, 33, 44, 55, 66);
 4   i  NUMBER;
 5 BEGIN
 6   t.DELETE(2);       -- Delete second element
 7   t(2) := 2222;      -- Restore second element
 8   t.DELETE(2, 4);    -- Delete range of elements
 9   i := t.FIRST;
10   WHILE i IS NOT NULL LOOP
11     dbms_output.put_line(t(i));
12     i := t.NEXT(i);
13   END LOOP;
14   t(3) := 3333;      -- Restore third element
15   t.DELETE;          -- Delete all elements
16 END;
17 /
```



# Collection Methods
## DELETE Example

```
 1 DECLARE
 2   TYPE nt is TABLE OF NUMBER(4) INDEX BY VARCHAR2(10);
 3   t nt := nt('a'=>1, 'b'=>2, 'c'=>30, 'd'=>4, 'e'=>5, 'f'=>6);
 4   i VARCHAR2(10);
 5 BEGIN
 6   t('c') := 300;  -- Replace element
 7   t('g') := 7; -- Add a new element
 8   t.DELETE('c');
 9   t.DELETE('g');
10   t('c') := 3; -- Replace deleted element with new value
11   t('g') := 7; -- Replace deleted element with same value
12   t('h') := 7; -- Add a new element
13   t('i') := 7; -- Add a new element
14   t('j') := 7; -- Add a new element
15   t.DELETE('c','e');  -- Delete range of elements
16   t.DELETE('j','f');  -- Does nothing
17
18   i := t.FIRST;
19   WHILE i IS NOT NULL LOOP
20     dbms_output.put_line(i||' '||t(i));
21     i := t.NEXT(i);
22   END LOOP;
23
24   t.DELETE; -- Delete all elements
25 END;
26 /
```



# Collection Methods
# TRIM

- **TRIM** is a procedure that deletes elements from the end of a varray or nested table

- This method has these forms:
  - **TRIM** removes one element from the end of the collection
    - If the collection has at least one element
    - Otherwise, it raises the predefined exception **SUBSCRIPT_BEYOND_COUNT**
  - **TRIM(n)** removes **n** elements from the end of the collection
    - If there are at least n elements at the end
    - Otherwise, it raises the predefined exception **SUBSCRIPT_BEYOND_COUNT**

- **TRIM** operates on the internal size of a collection:
  - **TRIM** considers the element to exist if **DELETE** deletes an element but keeps a placeholder for it
  - Therefore, **TRIM** can delete a deleted element

- PL/SQL does not keep placeholders for trimmed elements:
  - Trimmed elements are not included in the internal size of the collection
  - A trimmed element cannot be restored by assigning a valid value to it



# Collection Methods
## TRIM Example

```
 1 DECLARE
 2   TYPE nt is TABLE OF NUMBER(4);
 3   t nt := nt(1, 2, 3, 4, 5, 6);
 4   i  NUMBER;
 5 BEGIN
 6   t.TRIM; -- Trim last element
 7   t.DELETE(4); -- Delete fourth element
 8   t.TRIM(2); -- Trim last two element
 9   i := t.FIRST;
10   WHILE i IS NOT NULL LOOP
11     dbms_output.put_line(t(i));
12     i := t.NEXT(i);
13   END LOOP;
14 END;
15 /
```



# Collection Methods
## EXTEND

- **EXTEND** is a procedure that **adds elements** to the end of a **varray** or **nested table**

- The collection can be empty, but not null:
  - Use a constructor to make a collection empty or add elements to a null collection

- The **EXTEND** method has these forms:
  - **EXTEND** appends one null element to the collection
  - **EXTEND(n)** appends **n** null elements to the collection
  - **EXTEND(n,i)**
    - Appends **n** copies of the **i-th** element to the collection
    - Is the only form that you can use for a collection whose elements have the NOT NULL constraint

- EXTEND operates on the internal size of a collection:
  - EXTEND considers the element to exist if DELETE deletes an element but keeps a placeholder for it



# Collection Methods
## EXTEND Example

```
 1 DECLARE
 2   TYPE nt is TABLE OF NUMBER(4);
 3   t nt := nt(1, 2, 3, 4, 5, 6);
 4   i  NUMBER;
 5 BEGIN
 6   t.EXTEND(2, 3); -- Append two copies of first element
 7   t.DELETE(8); -- Delete eighth element
 8   t.EXTEND; -- Append one null element
 9   i := t.FIRST;
10   WHILE i IS NOT NULL LOOP
11     dbms_output.put_line(i||' '||t(i));
12     i := t.NEXT(i);
13   END LOOP;
14 END;
15 /
```



# Collection Methods
## EXISTS

- **EXISTS** is a function that tells you whether the specified element of a varray or nested table exists

- **EXISTS(n)**
  - Returns **TRUE** if the **n**-th element of the collection exists
  - Returns **FALSE** otherwise

- If **n** is out of range **EXISTS** returns **FALSE** instead of raising the predefined exception **SUBSCRIPT_OUTSIDE_LIMIT**

- **EXISTS(n)** returns **FALSE** for a deleted element (even if **DELETE** kept a placeholder for it)



# Collection Methods
## EXISTS Example

```
 1 DECLARE
 2   TYPE tbl1 IS TABLE OF countries.country_id%type;
 3   t1 tbl1 := tbl1('RO', 'MD');
 4   TYPE tbl2 IS TABLE OF countries.country_name%type;
 5   t2 tbl2 := tbl2('Romania', 'Moldova');
 6 BEGIN
 7   FORALL i IN INDICES OF t1 INSERT INTO countries(country_id) VALUES(t1(i));
 8
 9   FORALL i IN INDICES OF t2 UPDATE countries SET country_name=t2(i), region_id=1 WHERE country_id = t1(i);
10
11   SELECT country_id, country_name
12   BULK COLLECT INTO t1, t2 FROM countries
13   WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT NULL);
14
15   t1.DELETE(5, 7); -- Delete range
16
17   FOR i IN t1.FIRST .. t1.LAST
18   LOOP
19     IF t1.EXISTS(i) THEN -- test if exists
20       dbms_output.put_line(t1(i)||' '||t2(i));
21     END IF;
22   END LOOP;
23
24   FORALL i IN INDICES OF t1 DELETE countries WHERE country_id = t1(i);
25
26 END;
27 /
```



# Collection Methods
## FIRST and LAST

- **FIRST** and **LAST** are functions

- If the collection has at least one element
  - **FIRST** return the indexes of the first element
  - **LAST** return the indexes of the last element

- **FIRST** and **LAST** ignore deleted elements even if **DELETE** kept placeholders

- **FIRST** and **LAST** return the same index if the collection has only one element

- **FIRST** and **LAST** return **NULL** if the collection is empty



# Collection Methods
## FIRST and LAST

- For an associative array:
  - Indexed by **PLS_INTEGER**
    - The **FIRST** element is the smallest index
    - The **LAST** element is the largest index
  - Indexed by string
    - The **FIRST** element is  the lowest key value
    - The **LAST** element is  the highest key values
- For a varray that is not empty
  - **FIRST** always returns 1
  - **LAST** always equals **COUNT** for every varray
- For a nested table
  - **LAST** equals **COUNT** if elements from the middle of the nested table are **NOT** deleted
  - **LAST** is larger than **COUNT** if elements from its middle are deleted



# Collection Methods
## COUNT

- **COUNT** is a function that returns the number of elements in the collection
- **COUNT** ignores deleted elements, even if **DELETE** kept placeholders for them

- For a varray
  - **COUNT** always equals **LAST**
  - The value of **COUNT** changes if the size of the varray increases or decreases (with the **EXTEND** or **TRIM** method)

- For a nested table
  - **COUNT** equals **LAST** if elements from the middle of the nested table are not deleted
  - **COUNT** is smaller than **LAST** if elements from the middle of the nested table are deleted



# Collection Methods
## COUNT Example

```
1 DECLARE
2   TYPE rec IS RECORD(
3       country_name countries.country_name%TYPE,
4       region_name regions.region_name%TYPE
5     );
6   TYPE tbl IS TABLE OF rec;
7   t tbl;
8 BEGIN
9
10  SELECT country_name, region_name
11  BULK COLLECT INTO t
12  FROM countries natural join regions;
13
14  dbms_output.put_line(t.FIRST); -- 1
15  dbms_output.put_line(t.LAST); -- 25
16  dbms_output.put_line(t.COUNT); -- 25
17
18  t.DELETE(5, 15); -- Delete range
19
20  FOR i IN t.FIRST .. t.LAST
21  LOOP
22    IF t.EXISTS(i) THEN -- test if exists
23      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
24    END IF;
25  END LOOP;
26
27  dbms_output.put_line(t.FIRST); -- 1
28  dbms_output.put_line(t.LAST); -- 25
29  dbms_output.put_line(t.COUNT); -- 14
30
31 END;
32 /
```



# Collection Methods
## LIMIT

- **LIMIT** is a function that returns the maximum number of elements that the collection can have
- **LIMIT** returns **NULL** if the collection has no maximum number of elements
- Only a varray has a maximum size



# Collection Methods
## LIMIT Example

```
DECLARE
  TYPE tbl IS TABLE OF countries.country_name%TYPE;
  TYPE vec IS VARRAY(100) OF regions.region_name%TYPE;
  TYPE rec IS RECORD(
      country_names tbl,
      region_names vec
    );
  r rec;
BEGIN

  SELECT country_name
  BULK COLLECT INTO r.country_names
  FROM countries;

  SELECT region_name
  BULK COLLECT INTO r.region_names
  FROM regions;

  dbms_output.put_line('country_names.FIRST '||r.country_names.FIRST); -- 1
  dbms_output.put_line('country_names.LAST '||r.country_names.LAST); -- 25
  dbms_output.put_line('country_names.COUNT '||r.country_names.COUNT); -- 25
  dbms_output.put_line('country_names.LIMIT '||r.country_names.LIMIT); -- NULL

  dbms_output.put_line('region_names.FIRST '||r.region_names.FIRST); -- 1
  dbms_output.put_line('region_names.LAST '||r.region_names.LAST); -- 4
  dbms_output.put_line('region_names.COUNT '||r.region_names.COUNT); -- 4
  dbms_output.put_line('region_names.LIMIT '||r.region_names.LIMIT); -- 100

  FOR i IN r.country_names.FIRST .. r.country_names.LAST
  LOOP
    IF r.country_names.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(r.country_names(i));
    END IF;
  END LOOP;
END;
/
```



# Collection Methods
## PRIOR and NEXT

- PRIOR and NEXT
  - Are functions that let you move backward and forward in the collection
  - Ignore deleted elements, even if DELETE kept placeholders for them
  - Are useful for traversing sparse collections

- Given an index (the given index must exist):
  - PRIOR
  - If an index exists then PRIOR returns the index of the preceding existing element of the collection
    - If an index does not exist then PRIOR returns NULL
    - c.PRIOR(c.FIRST) returns NULL for any collection c
  - NEXT
    - If an index exists then NEXT returns the index of the succeeding existing element of the collection
    - If an index does not exist thenNEXT returns NULL
    - c.NEXT(c.LAST) returns NULL for any collection c



# Collection Methods
## PRIOR and NEXT

- If the collection c is a varray and the index exceeds c.LIMIT then:
  - c.PRIOR(index) returns c.LAST
  - c.NEXT(index) returns NULL

- For an associative array indexed by string
  - The prior and next indexes are determined by key values
  - Key values are in sorted order



# Collection Methods
## PRIOR and NEXT Example

```
DECLARE
    TYPE column IS TABLE OF INTEGER INDEX BY PLS_INTEGER;
    TYPE row IS TABLE OF column;
    mat row;
    tmat row := row();
    l STRING(100);
    i PLS_INTEGER;
    j PLS_INTEGER;
BEGIN
    mat := row(column(1=>1, 2=> 2, 3=> 3, 4=> 4),
               column(1=>5, 2=> 6, 3=> 7, 4=> 8),
               column(1=>9, 2=>10, 3=>11, 4=>12));

    i := mat.LAST;
    WHILE i IS NOT NULL LOOP
        j := mat(i).LAST;
        IF tmat IS EMPTY THEN
            tmat.extend(j);
        END IF;
        WHILE j IS NOT NULL LOOP
            tmat(j)(i) := mat(i)(j);
            j := mat(i).PRIOR(j);
        END LOOP;
        i := mat.PRIOR(i);
    END LOOP;

    dbms_output.put_line('Transpose Matrix');
    i := tmat.FIRST;
    WHILE i IS NOT NULL LOOP
        l := '';
        j := tmat(i).FIRST;
        WHILE j IS NOT NULL LOOP
            l := l || tmat(i)(j) || ' ';
            j := tmat(i).NEXT(j);
        END LOOP;
        dbms_output.put_line(l);
        i := tmat.NEXT(i);
    END LOOP;
END;
/
```



# Stored composite data types

```
 1 CREATE OR REPLACE TYPE employees_vec AS VARRAY(20) OF varchar2(20);
 2 /
 3
 4 CREATE TABLE EmployeesDeparments(
 5   numedept varchar2(30),
 6   angajati Employees_vec
 7 );
 8
 9 DECLARE
10   emps employees_vec;
11   idd departments.department_id%TYPE := 110;
12   dname departments.department_name%TYPE;
13 BEGIN
14   SELECT department_name INTO dname FROM departments WHERE department_id = idd;
15
16   SELECT first_name||' '||last_name
17   BULK COLLECT INTO emps
18   FROM employees
19   WHERE department_id = idd;
20
21   INSERT INTO EmployeesDeparments values(dname, emps);
22 END;
23 /
24
25 SELECT * FROM EmployeesDeparments;
26
27 DROP TABLE EmployeesDeparments;
28
29 DROP TYPE employees_vec;
```



# Stored composite data types

```
1  CREATE OR REPLACE TYPE jobs_nt AS TABLE OF varchar2(30);
2  /
3
4  CREATE TABLE EmployeesJobs(
5    ename varchar2(40),
6    ejobs jobs_nt
7  )
8  NESTED TABLE ejobs STORE AS ejobs_store;
9
10 set serveroutput on;
11 DECLARE
12   ejobs jobs_nt;
13   ide employees.employee_id%TYPE := 101;
14   ename varchar2(40);
15 BEGIN
16   SELECT first_name||' '||last_name INTO ename
17   FROM employees WHERE employee_id = ide;
18
19   WITH
20     q1 as (SELECT e.employee_id, j.job_title
21            FROM employees e
22            INNER JOIN job_history jh ON jh.employee_id=e.employee_id
23            INNER JOIN jobs j ON j.job_id = jh.job_id
24            UNION
25            SELECT e.employee_id, j.job_title
26            FROM employees e
27            INNER JOIN jobs j ON j.job_id = e.job_id)
28   SELECT job_title BULK COLLECT INTO ejobs
29   FROM q1
30   WHERE q1.employee_id = ide;
31
32   INSERT INTO EmployeesJobs values(ename, ejobs);
33 END;
34 /
35
36 SELECT * FROM EmployeesJobs;
37
38 DROP TABLE EmployeesJobs;
39
40 DROP TYPE jobs_nt;
```